1 What Is an Embedded System

The word *"system"* comes from physics: a system is a set of elements inside a boundary. Everything inside the boundary is the system; everything outside is its *environment*. To describe an embedded system, the first step is therefore to decide **what is inside the boundary** (components, functions) and what is **outside** (the physical world it observes and controls).

There is no single official definition of an embedded system. Three common definitions stress different aspects, and together they give a complete picture.

View	Definition	Key idea
Hardware and software	A combination of hardware and software (firmware) where the software is embedded within the hardware	The software cannot be written without knowing the hardware
Dedicated computer	A system based on a microcontroller or microprocessor that performs one specific task	Hardware and software are designed together for one job
Cyber-physical system	Computer hardware, software, and physical parts that perform a dedicated function	The computer interacts with the physical world

The third view explains what an embedded system usually does. *Sensors* measure physical quantities such as temperature, speed, or pressure. The processor runs the firmware and computes a response. *Actuators* turn that response into a physical action, such as moving a motor, opening a valve, or changing a display. Many embedded systems are also connected to other systems through a **network**.

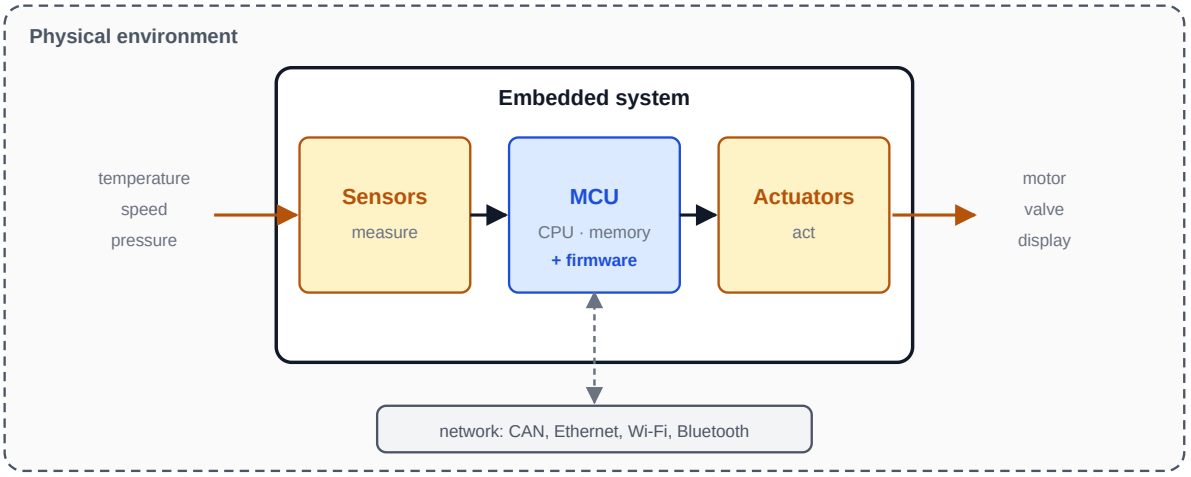


Figure 0.1: *An embedded system inside its environment. Sensors bring physical quantities in, the microcontroller runs the firmware, and actuators act on the world. A network may connect it to other systems.*

1.1 Classes of Embedded Systems

Embedded systems are used in very **different fields**, so they are **classified** in several ways. The three most useful classes are based on autonomy, timing, and connectivity. They are not exclusive: a car's brake controller is both real-time and networked.

Class	Main property	Example
Standalone	Works alone, without a network	Early washing machines, microwave ovens, thermostats
Real-time	Must complete each task before a deadline	Electronic brake control, drone stabilisation
Networked	Exchanges data with other nodes over a network or field bus	Engine, brake, and lighting controllers on a car's CAN bus

1.2 Embedded Versus General-Purpose Computers

1. A **general-purpose computer**, such as a laptop, runs any program the user installs, supports many users and tasks, and has plenty of memory and power.
2. An embedded system runs **one fixed job** and works under **strict limits**.

Aspect	Embedded system	General-purpose computer
Workload	One fixed task	Anything the user installs
Resources	KB to a few MB of RAM and Flash	GB of RAM, TB of disk
Timing	Often hard deadlines	Best effort
User model	No interactive user, no shell	Multi-user, multi-tasking
Power and cost	Milliwatts, cents to a few euros	Tens to hundreds of watts, hundreds of euros

These limits change **how software is written**. Embedded software cannot rely on an operating system to create a process, assign an address space, discover devices, or provide standard input and output. It is **built for one known processor** and one known memory map. The hardware details are visible and cannot be ignored, but this does not make the software simple: the developer must handle many tasks that a desktop OS normally hides.

The border between the two worlds is becoming less sharp. The same processor families, especially ARM, now range from tiny sensor nodes to smartphones, laptops, and cloud servers.

1.3 A Short History

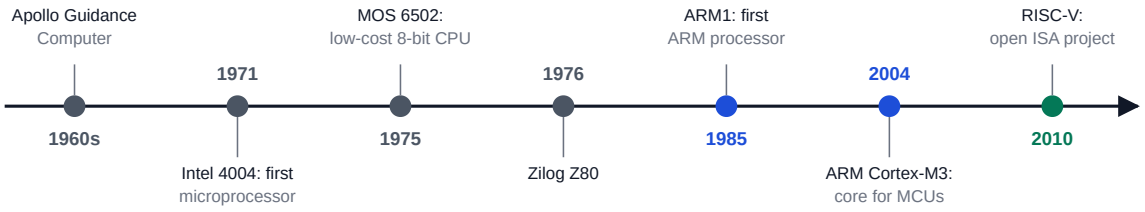


Figure 0.2: *Milestones of embedded computing, from the Apollo Guidance Computer to the open RISC-V instruction set (not to scale).*

Embedded computing grew from dedicated machines such as the Apollo Guidance Computer to cheap single-chip

processors, and today it is dominated by ARM cores, with the open RISC-V instruction set as a newer alternative. Embedded products also have a long lifecycle: a satellite or an industrial machine may keep the same processor for ten or twenty years, so reliability and **long-term support** often **matter more than performance**.

2 Design Constraints of Embedded Systems

Embedded systems are designed under **constraints** that rarely matter on a desktop. Each one shapes both the hardware and the software.

Constraint	Why it matters	Example
Real-time	Results must arrive be**fore a deadline**	A brake controller must react within a few milliseconds
Size and weight	Extra mass costs energy and space	A heavier controller shortens a drone's flight time
Cost	Small extra costs multiply over millions of units	A 50-cent security chip rejected for a car because of the total cost
Energy	Many devices run on batteries	A wearable or remote sensor must last months on one battery
Reliability and safety	Failures can damage machines or hurt people	Medical devices and robots must *fail safe*
Development time	Low-level code is slow to write and test	Hardware debugging and testing in all operating conditions

Fail safe means that when a fault is detected, the system moves to a **controlled and predictable state** instead of acting in an uncontrolled way. In a safety-critical system, every failure scenario must be considered at design time.

2.1 Real-Time Means Bounded Response

Real-time does not mean fast. A real-time system guarantees a **bounded and predictable** response time: every reaction to an event must complete before its **deadline**. A correct embedded system must therefore meet three kinds of requirements.

Requirement	Example
Functional	Sample a sensor correctly
Temporal	Sample it every 1 ms, before the deadline
Robustness	Continue safely when one input is missing

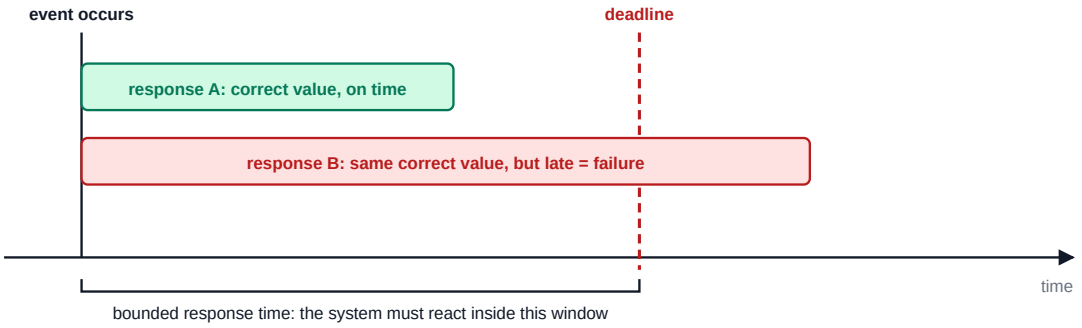


Figure 0.3: *Two responses to the same event. Both compute the correct value, but only the first arrives before the deadline; the second is a failure.*

In a *hard* real-time system, missing a deadline is a failure of the system, as in a braking controller. In a *soft* real-time system, a late result is still useful but lowers the quality of service, as in video playback.

> ⚠ **CORRECT VALUE, WRONG TIME**
>
> A result computed after its deadline can be a failure even if its value is correct. A fast system is not necessarily a real-time system, because average speed says nothing about the worst case.

3 Inside the Hardware: Processor, Memory, and Buses

A computer has four basic parts:

1. Central processing unit (CPU),
2. memory,
3. input units,
4. output units.

An embedded developer works much closer to these parts than an application developer does, so a basic model of each is needed.

3.1 Microprocessors and Microcontrollers

The **core** of a microprocessor and the core of a microcontroller execute instructions in the **same way**. The difference is what surrounds the core.

A ***microprocessor*** is mainly the CPU. It needs external RAM, external storage, and external peripheral chips to form a working computer. At power-on it runs a small firmware stored in a boot memory. This firmware starts a *boot loader*, which copies the operating system from storage (a disk or an SD card) into RAM and jumps to it.

A ***microcontroller*** **(MCU)** puts the CPU core, RAM, Flash, and peripherals (timers, converters, serial interfaces) on a **single chip**. Its internal Flash is mapped at the address the CPU reads after reset, so the firmware starts running directly from Flash, without a boot loader.

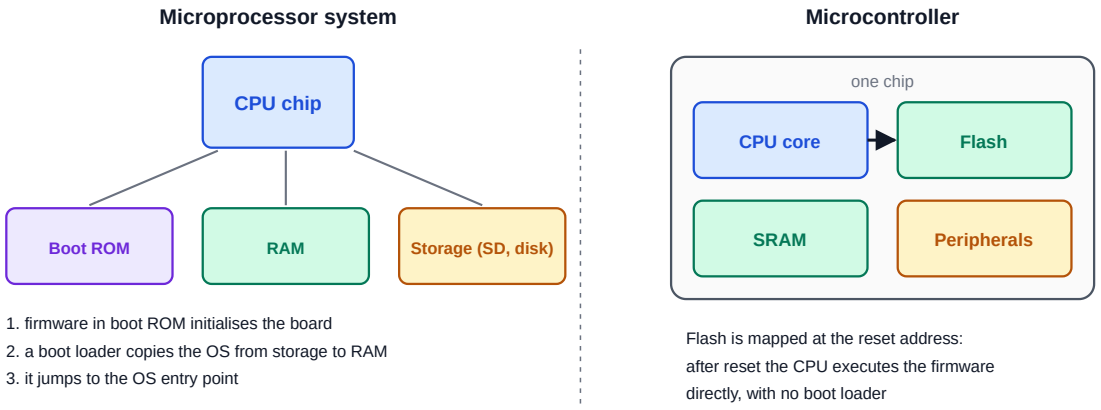


Figure 0.7: *A microprocessor needs external memories and a boot loader; a microcontroller integrates everything and executes its firmware straight from Flash after reset.*

3.2 The CPU and the Pipeline

A CPU repeats the same cycle forever:

1. **Fetch**: it reads the next instruction from memory, at the address held in the program counter.
2. **Decode**: it works out what the instruction asks for: which operation, and which registers or memory address it uses.

3. **Execute**: it performs the operation, for example an addition in the ALU, a load or store to memory, or a jump to another address.

Inside the CPU, the *control unit* drives this cycle, the *arithmetic logic unit* (ALU) performs arithmetic and logic operations, and *registers* hold the values being processed.

Registers are the fastest storage in the system, and a few of them have special roles:

1. **Program counter** (`PC`): holds the address of the next instruction to fetch. It advances automatically after each instruction; a branch or a function call changes it to jump elsewhere.
2. **Stack pointer** (`SP`): holds the address of the top of the stack, the memory area that stores local variables, return addresses, and saved registers during function calls and interrupts.
3. **Status register**: holds flags that describe the result of the last operation (for example zero, negative, carry, overflow), which conditional branches use to implement `if` statements and loops. It can also hold information about the processor state, such as whether interrupts are enabled.

Modern CPUs overlap these steps with a ***pipeline***. Each step is handled by a different part of the hardware, so while one instruction executes, the next one is decoded and a third one is fetched, like cars in a car wash with one worker per station.

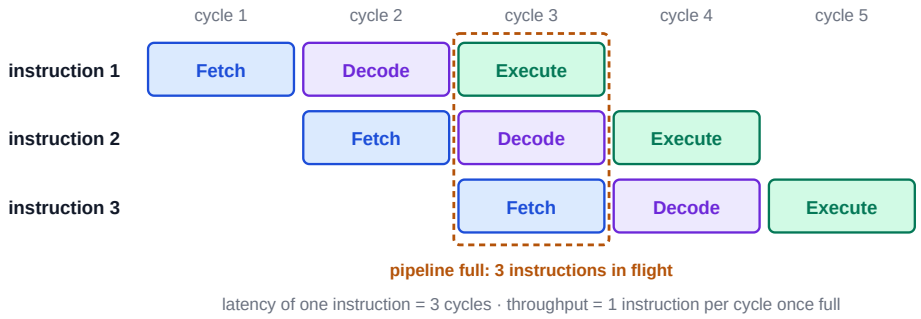


Figure 0.4: *A three-stage pipeline. From cycle 3 on, three instructions are in flight and one instruction completes every cycle.*

With n stages of duration t, a single instruction still needs $n \cdot t$ (its *latency* does not change), but once the pipeline is full one instruction completes every t. A sequence of m instructions takes

$$T_{total} = (n + m - 1) \cdot t$$

So pipelining increases *throughput*, not the speed of each instruction. The Cortex-M3 uses a three-stage pipeline of this kind.

3.3 Von Neumann and Harvard Architectures

CPU architectures differ in how they reach instructions and data. In the *von Neumann* architecture, instructions and data share one memory and one bus. It is simple and flexible, but the **shared bus limits** how fast data can move (the *von Neumann bottleneck*).
In the *Harvard* architecture, instructions and data use separate memories and buses, so the CPU can fetch an instruction and access data at the same time.

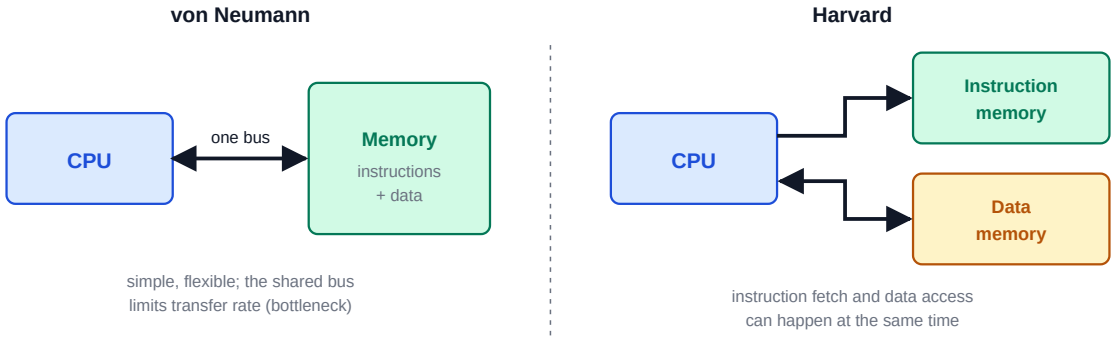


Figure 0.5: *von Neumann (one memory, one bus) and Harvard (separate instruction and data paths). The Cortex-M3 mixes the two: separate buses, one address space.*

Real processors often mix the two models. The Cortex-M3 has separate buses for instruction fetches and data accesses, which gives Harvard-like performance, but code and data share one 4 GB address space, as in von Neumann.

3.4 RISC and the Load/Store Model

ARM processors follow the **RISC (Reduced Instruction Set Computer)** philosophy: a small set of simple instructions, each executed quickly, combined with good compilers.
A key RISC rule is the ***load/store*** model: arithmetic and logic work **only on registers**, so a value must be loaded from memory into a register, processed, and stored back.
CISC (Complex Instruction Set Computer) processors, such as Intel *x86*, can instead **operate directly on memory**. Compilers hide this difference in C, but it becomes visible in assembly and in operating-system code.

3.5 Memory Hierarchy and Memory Technologies

Memories differ in speed, size, and cost, so every computer arranges them in a hierarchy. Small and fast memories sit close to the CPU; large and slow memories sit further away.

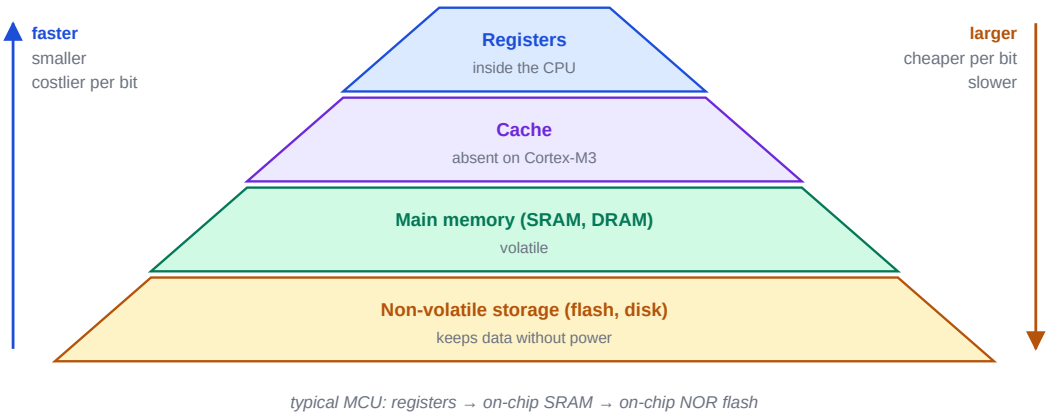


Figure 0.6: *The memory hierarchy. Going down, memories become larger and cheaper per bit but slower.*

A *cache* is a small, fast memory that automatically keeps recently used data close to the CPU. It improves

average speed, but its access time depends on what happened before, so the **same code can take different times** on different runs. This makes **timing hard to predict**, which is why many **real-time systems disable caches** or configure them carefully.

> **IMPORTANT**
>
> The Cortex-M3 core has no cache at all.

3.6 Buses and Peripherals

A *bus* is a set of **shared wires** that moves information between parts of a computer. In the textbook model, one system bus joins the CPU and the memories and is made of **three groups of lines**:

1. **address lines** select the memory location or device,
2. ***data lines*** carry the value being read or written,
3. ***control lines*** say whether the operation is a read or a write and when the data is valid.

A real microcontroller uses a **hierarchy of buses** instead of a single one. A fast on-chip bus (on ARM chips, the **AHB**) connects the CPU core with Flash and SRAM, which must be reached at every cycle.
Peripherals are slower and far more numerous, so they sit on a separate, simpler peripheral bus (the **APB**), reached through a *bridge*. This split keeps the fast bus small and saves power. The buses of the `mps2-an385` board are described in the chapter on devices, interrupts, and system calls.

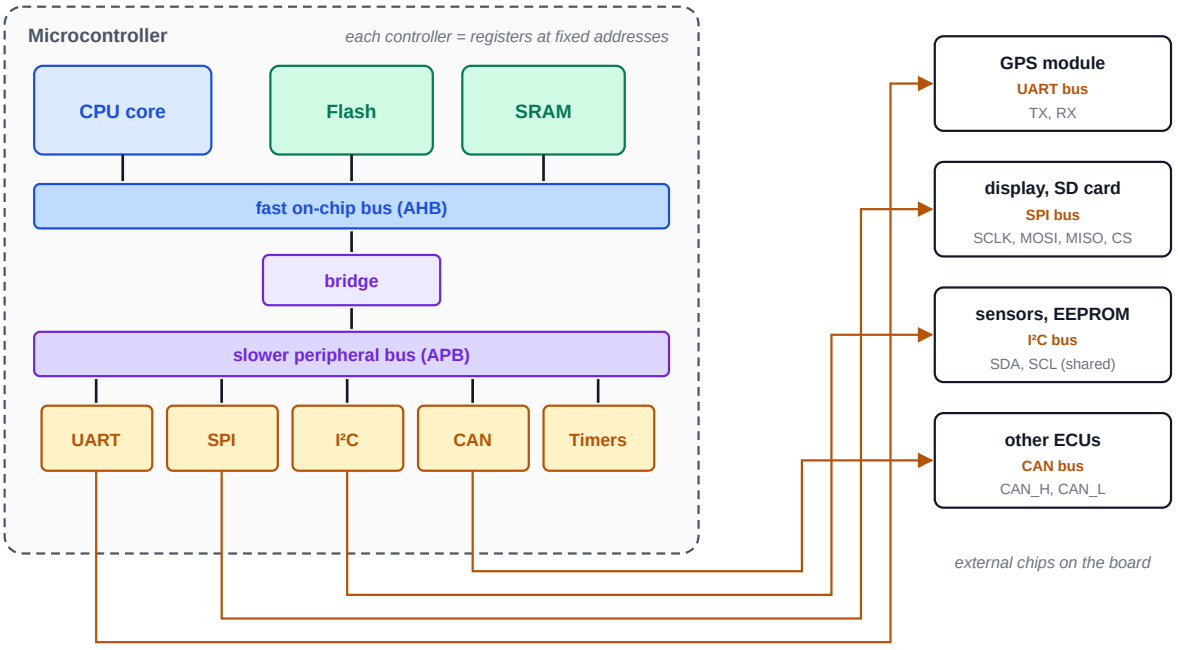


Figure 0.7: *Buses of a microcontroller. A fast on-chip bus connects the CPU and memories, a bridge leads to a slower peripheral bus, and each peripheral controller drives a serial bus to external chips.*

Each on-chip peripheral is a *controller* that the CPU programs through its *registers*, placed at fixed addresses in the memory map. Firmware configures a controller by writing these registers and checks its state by reading them, following the rules in the chip's datasheet. Many controllers drive a *serial bus*, which sends bits one after another over a few wires, to talk to other chips on the board.

On a desktop, device drivers inside the operating system hide all of this behind files and system calls. In an embedded system the firmware usually talks to the controllers directly, which is why register-level programming is a central skill of embedded development.

4 Operating Systems

An **operating system (OS)** has **several** classic **definitions**, each stressing a different role.

Definition	Emphasis
Intermediary	A middle layer between hardware and applications that hides hardware details
Resource manager	Software that shares CPU, memory, I/O devices, and storage among applications
Master control program	The first software loaded at power-on; its core, the *kernel*, stays in memory

ROLE OF AN OPERATING SYSTEM

An OS mediates between application software and hardware. It gives controlled access to processors, memory, devices, and time, and it decides how competing work is scheduled.

The OS has two main purposes:

1. First, it **makes the hardware usable**: without an OS, every application would have to initialise the CPU, program timers, share communication links, and handle interrupts on its own.
2. Second, it **manages resources** that are **limited** and **shared**, even on a small board: CPU time (which task runs now), memory (which code and data occupy RAM), devices (who may use a timer or read a received byte), and events (how an interrupt becomes work for the right task).

Internally, an OS can be seen as a set of *managers*, one per resource, placed above a layer of hardware-specific services. Applications never call the managers directly: they go through a single *system-call interface*.

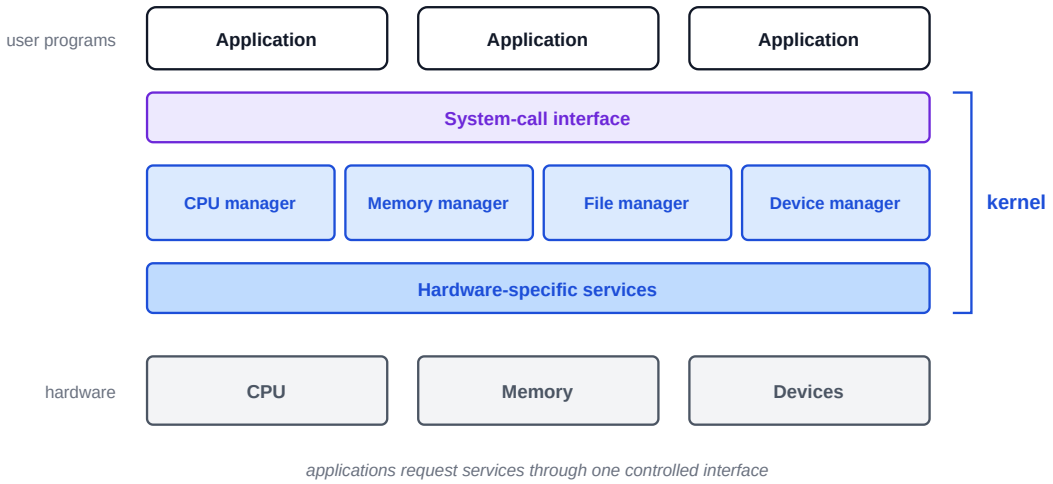


Figure 0.8: *The structure of an operating system. Applications use the system-call interface; the kernel's managers coordinate the hardware through hardware-specific services.*

4.1 Operating System Services

The services of a general-purpose OS fall into **three groups**.

Group	Services
For users	User interface (command line, graphical, touch, batch); program execution and termination
For programs	Uniform I/O operations through device drivers; file-system operations; communication between processes
For safe sharing	Resource allocation; logging and accounting; protection and security; error detection and handling

An **embedded OS** usually offers only a **small part of this list**. A microcontroller OS may provide only task scheduling, synchronisation, and time services, while the application keeps its own device drivers.

> **THE OS BOUNDARY IS A DESIGN CHOICE**
>
> What belongs to the OS is not a fixed feature list. It depends on what the product needs.

4.2 Privilege Levels, Traps, and System Calls

To protect itself and the hardware, an OS relies on CPU *execution modes*:

- **User mode (restricted)**: the mode of application code. A program in user mode can compute and access its own memory, but it cannot program I/O controllers, change memory-protection settings, or disable interrupts. If it tries, the CPU blocks the operation: depending on the processor and on the instruction, it raises an exception or silently ignores it.
- **Kernel mode (privileged)**: the mode of the kernel, where every instruction and every register is allowed. Only code in kernel mode can configure devices, memory protection, and interrupts.

> **IMPORTANT**
>
> The CPU switches from user to kernel mode only in controlled ways, through an **exception**, an **interrupt**, or a **system call**, so an application can never decide by itself to run with privileges.

When an application needs a privileged operation, it makes a ***system call***. A system call is implemented with a ***trap***, a special instruction that moves execution from user mode to kernel mode in a controlled way. The CPU switches mode, **saves the context** of the program (for example the program counter and some registers), and jumps to a fixed kernel entry point. The kernel checks the arguments, performs the service, prepares a return value, restores the context, and returns to user mode.

A SYSTEM CALL FROM C

The following program, written for a PC running Linux, prints a message on the terminal with a single system call.

```
#include <unistd.h>   /* write() */
#include <string.h>   /* strlen() */


int main( void )
{
    const char *msg = "Greetings\n";
    ssize_t n = write( 1, msg, strlen( msg ) );  /* 1 = standard output */
    return ( n < 0 ) ? 1 : 0;
}
```

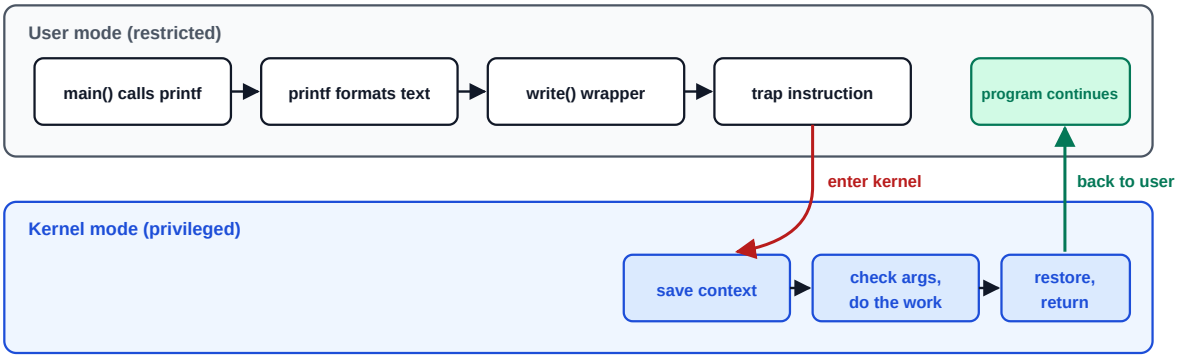
The call `write( 1, msg, strlen( msg ) )` asks the kernel to write some bytes to an output. Its three arguments say *where* to write, *what* to write, and *how many* bytes:

- `1` is a *file descriptor*, a small number that identifies an open file or device; descriptor 1 is the standard output, normally the terminal;
- `msg` is the address of the bytes to write;
- `strlen( msg )` is their number, here 10.

The program never touches the terminal itself, because in user mode it is not allowed to. The function `write` from the C library is only a thin *wrapper*: it places the service number and the three arguments in the registers where the kernel expects them and executes the trap. The kernel checks the request, its terminal driver prints the bytes, and control comes back to the program.

`write` returns the number of bytes actually written, or −1 if the operation failed. The program uses this value as its exit code: 0 means success, 1 means error.

Programs rarely execute the trap instruction themselves. They call library functions such as `printf`, which formats the text in user mode and then calls `write`, a thin *wrapper* that places the arguments where the kernel expects them and executes the trap. The return value tells the program whether the operation succeeded.



the same shape on Cortex-M: svc enters a privileged handler, exception return goes back

Figure 0.9: *A system call. The program reaches the trap through the C library, the kernel does the work in privileged mode, and control returns to the program in user mode.*

The Cortex-M3 implements the same mechanism with the `svc` instruction and the privileged Handler mode, as described in the chapter on the ARM Cortex-M3 architecture.

4.3 From Source Code to a Running OS

A program becomes **executable** in three steps:

1. The *compiler* turns each source file into a *relocatable object file*, whose final addresses are not fixed yet.
2. The *linker* combines the object files and libraries into one executable image (an ELF file), resolves the references between them, and assigns final addresses.
3. Finally, the image must be placed in memory and started: on a PC this is the job of a *loader*, which can also relocate the program to a different address.

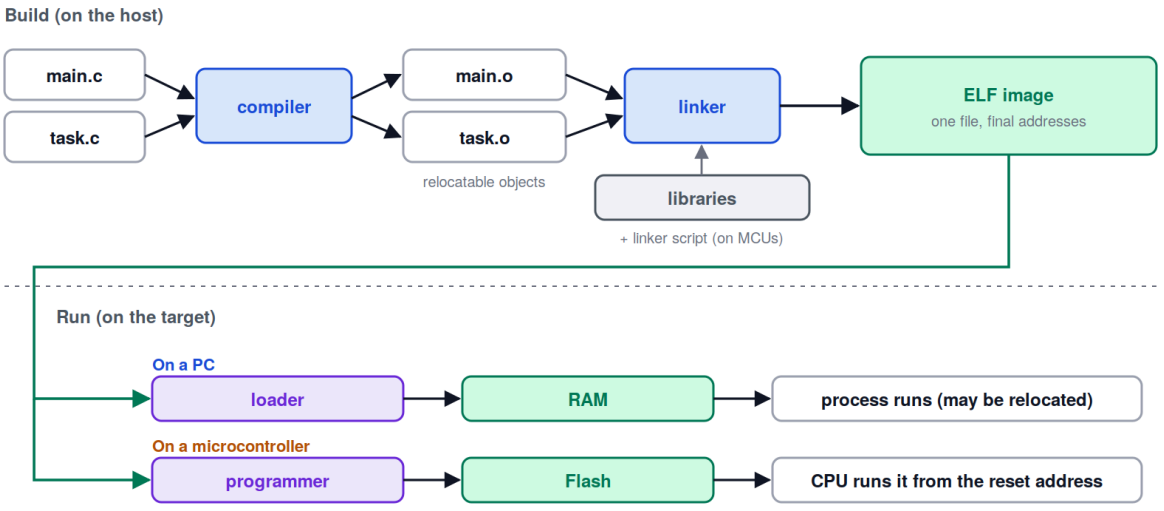


Figure 0.11: *From source code to a running program. The compiler and linker run on the host and produce one ELF image; a PC loads it into RAM with a loader, while a microcontroller has it written into Flash and runs it from the reset address.*

On a microcontroller the OS image is written into Flash with an external programmer during development, and the CPU runs it from its reset address.

Applications can be added to such a system in two ways:

- With ***fixed tasks***, the tasks are compiled and linked together with the OS into one image, and **no loader is needed**.
- With ***programmable tasks***, applications are separate images loaded after the OS starts; this allows updates without rebuilding the OS, at the cost of a loader, memory management, and security checks.

Most small embedded systems use **fixed tasks**.

5 Kernel Architectures

Operating systems differ in *where* their services run and *how strongly* they are isolated from each other and from applications.

Architecture	Where services run	Advantage	Cost	Examples
Flat	Everything in one image and one address space	Small and fast	Little or no isolation	FreeRTOS, MS-DOS
Monolithic	All services in one privileged kernel	Fast direct calls between services	Large trusted core; a bad driver can crash the kernel	Linux, UNIX
Microkernel	Minimal kernel; services as user-mode servers	Small trusted base, faults contained	IPC overhead, more interfaces	seL4, QNX
Hybrid	Mix of the above	Balance of speed and structure	Complexity	macOS (Mach + BSD), Linux modules

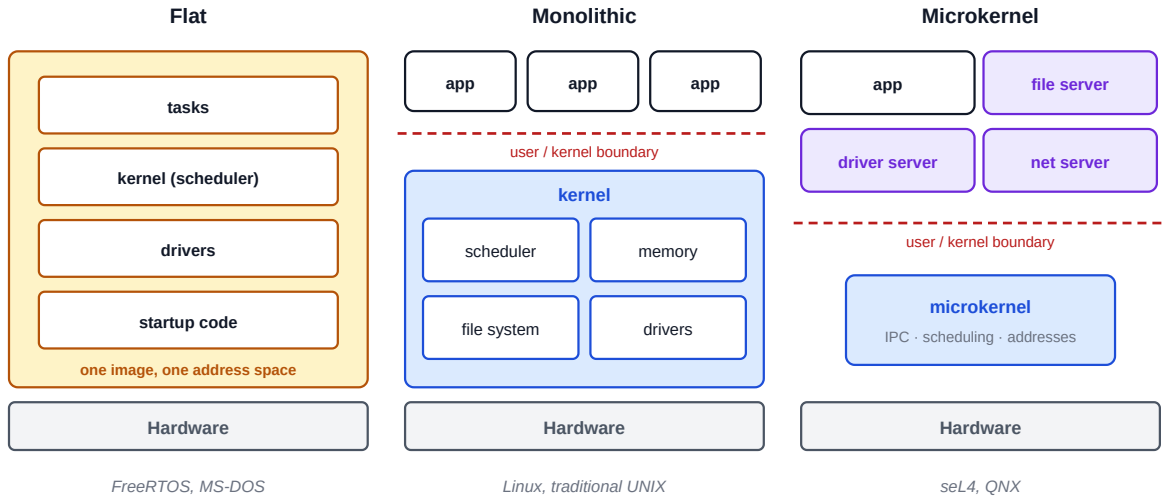


Figure 0.10: *Three kernel architectures. The red line is the hardware-enforced boundary between user and kernel mode; a flat system has none.*

1. In a ***flat* design**, startup code, kernel, drivers, and tasks are linked into one image and share one physical address space. OS services are ordinary functions that tasks call directly. No memory management unit and no privilege levels are needed, but any bug can corrupt any data.
2. A ***monolithic* kernel** separates user space from kernel space with hardware support (privilege levels and an **MMU** or **MPU**), so a **faulty application cannot corrupt the kernel**. Inside the kernel, however, scheduler, memory manager, file systems, and drivers share the same privileged space. Drivers are a large and error-prone part of the code, and a single faulty driver can crash the whole system.

3. A ***microkernel*** keeps only the **basic mechanisms in kernel mode**, typically scheduling, address-space management, and inter-process communication (IPC). File systems, drivers, and network stacks run as user-mode servers that exchange messages. A failed server can be restarted, and the small kernel is easier to verify. The **price is performance**: a system call in a monolithic kernel needs two mode switches (user to kernel and back), while the same request in a microkernel typically needs four (client to kernel, kernel to server, server to kernel, kernel to client).

Most real systems are *hybrids*. Linux is monolithic but loads drivers as modules at run time; macOS combines the Mach microkernel with BSD components.

> **CHOOSING A KERNEL ARCHITECTURE**
>
> Each architecture is a trade-off between isolation, fault containment, overhead, and complexity. The useful question is not "which one is best?" but "what must this system guarantee?".

6 Embedded and Real-Time Operating Systems

Embedded OSes differ from desktop OSes for three reasons:

1. They are ***resource-constrained***: a few KB of RAM leave no room for a large kernel.
2. They are ***real-time***: correctness depends on when things happen.
3. They usually have ***no untrusted users***: one vendor ships one firmware image with a known set of tasks. For these reasons an embedded OS is selected and configured for one product, not for an unknown future workload.

General-purpose mechanism	Embedded replacement
Dynamic allocation	Static number of tasks and fixed-size buffers
General fairness policy	Simple priority-based scheduler
Layered device stack	Direct register access
Maximum throughput	Predictable execution time

> **SMALL IS NOT INCOMPLETE**
>
> A small OS is complete when it matches the responsibilities of the system.

A ***real-time operating system*** **(RTOS)** such as FreeRTOS provides ready-made versions of the mechanisms an embedded application needs: tasks with priorities, delays and timer-driven scheduling, queues, semaphores and mutexes, and communication from interrupts to tasks. By default FreeRTOS has no memory protection between tasks: like flat firmware, all tasks share one address space. Knowing the mechanisms underneath an RTOS makes its API much easier to use correctly.

6.1 Single Image, Single Failure Domain

Many microcontroller products ship one firmware image that contains startup code, drivers, OS, and application. **Updating the image changes the whole system**. This makes deployment simple, but it requires discipline: a bad pointer can damage any writable data, a blocked high-priority task can stop an important service, and recovery and diagnostics must be designed into the firmware from the start. Scheduling, synchronisation, and memory protection are the mechanisms that reduce these risks.

6.2 From Bare Metal to a Minimal OS

A minimal flat OS is built as a **chain of mechanisms**, where each one depends on the previous one. This chain is also the path followed by the rest of these notes, on an ARM Cortex-M3 microcontroller emulated by QEMU.

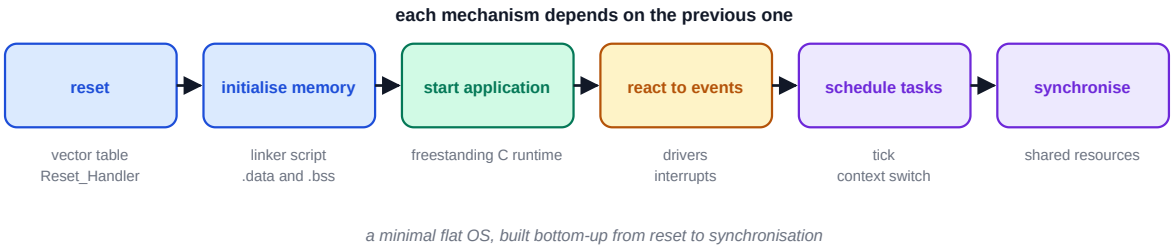


Figure 0.11: *The mechanisms of a minimal OS, from reset to synchronisation, with the main topics involved in each step.*

- **Reset**: the processor must start correctly after reset, which requires a vector table and a reset handler.
- **Initialise memory**: memory must be prepared as described by the linker script, copying `.data` and zeroing `.bss`, so that C code can run.
- **Start the application**: the freestanding C runtime calls `main()`, without any operating system underneath.
- **React to events**: the application reacts to the outside world through device drivers and interrupts.
- **Schedule tasks**: with a periodic timer tick and context switching, the kernel runs several tasks.
- **Synchronise**: the kernel coordinates the tasks' access to shared resources. Memory protection and system calls, built on the privilege levels of the Cortex-M3, add isolation on top of this chain.